Mobile UI Input Handling Improvements

SimFusion 3D Anatomy Viewer · Android Platform

Unity 6000.3.14 LTS · Universal Render Pipeline

Overview

On Android, interacting with UI elements — specifically the Section Slider, body system toggles, and mode buttons — could unintentionally rotate the camera. Dragging the Section Slider was the most reproducible case: the camera would orbit while the slider value was being adjusted, making the control difficult to use and degrading the overall mobile experience.

The issue was not a missing feature. A UI guard had been implemented as part of the initial camera input system. The problem was that the guard used a mechanism that is unreliable during drag operations on Android, meaning it correctly blocked input in simple cases (tapping a button) but failed silently in the case that mattered most (dragging the slider).

Root Cause

The unreliable per-frame check

The initial implementation called `EventSystem.current.IsPointerOverGameObject(fingerId)` on every `TouchPhase.Moved` frame to determine whether the camera should receive input:

```
if (IsOverUI(touch.fingerId))
    return;
```

This answers a positional question: *is this finger's current screen position over a UI element right now?* For button taps, the finger stays within the button's `RectTransform` and the check returns `true` reliably. For a slider drag, the check fails.

How `PointerExit` breaks the guard

Unity's `StandaloneInputModule` (the active input module in this project) tracks touch state per `PointerEventData`. When a finger moves outside a UI element's `RectTransform`, the module fires a `PointerExit` event for that element and clears `pointerEnter` to `null`. The `IsPointerOverGameObject` implementation checks `pointerEnter`:

```
IsPointerOverGameObject(fingerId)
  → GetLastPointerEventData(fingerId)
  → returns lastPointer.pointerEnter != null
```

Once `pointerEnter` is null, the check returns `false`. For a slider, the user's finger must travel beyond the slider handle's bounds to produce a meaningful range of values. The moment it crosses that boundary, `IsPointerOverGameObject` returns `false`, the camera guard is lifted, and the orbit delta is written — all while the slider drag is still active.

Camera input generated independently of UI interaction

`TouchCameraInput` and the UI system are two independent consumers of the same raw touch data. The EventSystem processes UI drag events through its own pipeline. `TouchCameraInput` reads `Input.GetTouch` directly in `Update()` and writes a cached delta that `OrbitCameraController` consumes in `LateUpdate()`. These pipelines have no shared state, so the camera had no way to know that a UI drag was in progress once the per-frame position check started returning `false`.

Implementation

Principle: track UI ownership, not UI position

The core insight from the investigation is that the wrong question was being asked. The per-frame check asks *where is the finger now?* The correct question is *did this finger start a UI interaction?* A touch that begins over a UI element belongs to the UI for its entire lifetime. Camera input from that finger should be suppressed from `TouchPhase.Began` through `TouchPhase.Ended` or `Canceled`, regardless of where the finger travels.

UI-owned touch registry

A `HashSet<int>` named `_uiTouches` stores the `fingerId` of every touch that began over a UI element. A dedicated method, `UpdateUITouchRegistry()`, runs at the start of every `Update()` frame and processes each active touch:

- `TouchPhase.Began`: call `IsPointerOverGameObject(fingerId)`. This is the one moment the EventSystem is guaranteed to return an accurate result — the finger just made contact and the `PointerEventData` has been freshly created and raycasted. If the result is `true`, the `fingerId` is added to `_uiTouches`.
- `TouchPhase.Moved` / `TouchPhase.Stationary`: no action in the registry. The answer established on `Began` is simply held.
- `TouchPhase.Ended` / `TouchPhase.Canceled`: the `fingerId` is removed from `_uiTouches`. Camera input from this finger is restored immediately on the next frame.

Three named helper methods wrap the `HashSet` operations:

```
private void RegisterUITouch(int fingerId)  => _uiTouches.Add(fingerId);
private void ReleaseUITouch(int fingerId)   => _uiTouches.Remove(fingerId);
private bool IsUIOwnedTouch(int fingerId)   => _uiTouches.Contains(fingerId);
```

These names communicate intent clearly. The gesture handlers call `IsUIOwnedTouch` rather than querying the EventSystem, so the question asked in the hot path is the correct one.

Single-touch handling

`HandleSingleTouch` checks `IsUIOwnedTouch` before computing an orbit delta. If the finger is registered, the method returns immediately. Because `_pinchActive` is cleared at the top of `HandleSingleTouch` regardless of UI state, releasing a UI touch never leaves the pinch state machine in an inconsistent state.

Two-finger gesture handling

`HandleTwoTouches` checks both fingers. If either belongs to `_uiTouches`, the gesture is aborted and `_pinchActive` is reset. This handles the case where the user begins a two-finger gesture with one finger on the viewport and one on a UI panel — the entire gesture is cancelled rather than producing a partial zoom or pan.

`_pinchActive` is reset on abort so that when the UI-owned finger is eventually released, the remaining viewport finger does not produce a position jump on the next `HandleTwoTouches` call; instead, the pinch baseline is re-established cleanly.

Localization

The entire implementation is contained within `TouchCameraInput`. `OrbitCameraController` consumes a cached delta via `ICameraInputProvider.GetOrbitDelta()` with no knowledge of where that delta came from or how it was gated. `CombinedCameraInput` sums the PC and touch providers with no knowledge of UI state. Neither class required modification.

Files Modified

`TouchCameraInput.cs`

- Added `using System.Collections.Generic` for `HashSet<int>`.
- Added `_uiTouches` field — the UI-owned touch registry.
- Added `UpdateUITouchRegistry()` — processes `Began` / `Ended` / `Canceled` phases each frame.
- Added `RegisterUITouch()`, `ReleaseUITouch()`, `IsUIOwnedTouch()` — named helpers for registry access.
- Renamed the per-frame positional check to `IsPointerOverUI()` to distinguish it clearly from the lifetime check.
- Updated `HandleSingleTouch()` to guard on `IsUIOwnedTouch` instead of `IsPointerOverUI`.
- Updated `HandleTwoTouches()` to guard on either finger's `IsUIOwnedTouch` state and reset `_pinchActive` on abort.
- Existing `CameraSettings` sensitivity values (`TouchOrbitSensitivity`, `TouchPinchSensitivity`, `TouchPanSensitivity`) preserved unchanged.

`PCCameraInput.cs`

- Added `using UnityEngine.EventSystems`.
- Added a per-frame `IsPointerOverGameObject()` check (mouse pointer, no finger ID) to suppress orbit, pan, and zoom while the mouse pointer is over a UI element.
- `_lastMousePos` is updated unconditionally, outside the UI guard. This ensures that when the pointer leaves a UI element, the first camera drag frame computes a delta against the current position rather than the stale position from before UI interaction, preventing a jump.

Design Decisions

Separation of concerns

Input generation (what the user is doing) and input gating (whether the camera should respond) are both responsibilities of the input provider. Placing the UI-ownership check in `TouchCameraInput` keeps the gate at the source of the delta. `OrbitCameraController` remains a pure consumer of normalised deltas with no UI awareness.

Minimal architectural impact

The fix adds one field, four methods, and one call site in `UpdateUITouchRegistry`. It does not introduce new MonoBehaviours, new interfaces, or new dependencies. The `ICameraInputProvider` contract is unchanged. The existing three-class camera pipeline (`TouchCameraInput` → `CombinedCameraInput` → `OrbitCameraController`) is preserved without modification.

O(1) operations

`HashSet<int>` provides constant-time `Add`, `Remove`, and `Contains`. The number of simultaneous touches is bounded by the hardware (typically five on Android). The set never needs to be iterated — only single-ID lookups are performed in the gesture handlers.

No runtime allocations after initialisation

`_uiTouches` is constructed once as a field initialiser and never replaced. `HashSet<int>` does not box integer keys (unlike `HashSet<object>`). `Add` and `Remove` on an existing `HashSet<int>` with capacity to spare do not allocate. The fix introduces zero garbage-collected objects per frame.

Maintainability over cleverness

The registry pattern makes the intent explicit at every call site. A future developer reading `IsUIOwnedTouch(touch.fingerId)` understands immediately that this is a lifetime question, not a position question. The alternative — a per-frame position check — requires knowing the `StandaloneInputModule` implementation detail to understand why it fails. The current implementation is self-documenting.

Performance Considerations

Android hardware imposes a maximum of five simultaneous touch points. `_uiTouches` will never hold more than five entries. The `HashSet<int>` pre-allocates a small internal array at construction; no resizing occurs during normal gameplay.

The cost per frame is bounded:

Operation	Where	Cost
`UpdateUITouchRegistry` loop	Once per `Update()`	O(touchCount) ≤ O(5)
`IsUIOwnedTouch` in gesture handlers	Once or twice per `Update()`	O(1)
`IsPointerOverGameObject`	Only on `TouchPhase.Began`	O(1), infrequent

`IsPointerOverGameObject` — which involves an EventSystem lookup — is now called only on `TouchPhase.Began`, not on every `Moved` frame. This reduces EventSystem queries to a small fraction of what the previous per-frame approach performed.

Total added cost relative to the original implementation is negligible and unmeasurable in profiling.

Testing Performed

Scenario	Expected behaviour	Result
Section Slider drag	Slider value changes; camera does not rotate	✓ Pass
Slider drag beyond slider bounds	Camera remains still while finger travels outside the slider rect	✓ Pass
Body System toggle tap	System toggles; camera does not rotate	✓ Pass
X-Ray button tap	X-Ray mode toggles; camera does not rotate	✓ Pass
Reset View button tap	Camera resets; no additional orbit input	✓ Pass
Orbit in viewport	Single-finger drag rotates camera normally	✓ Pass
Pinch zoom in viewport	Two-finger pinch zooms; no UI involved	✓ Pass
Two-finger pan in viewport	Two-finger drag pans; no UI involved	✓ Pass

Scenario	Expected behaviour	Result
One finger on UI, one in viewport	Entire two-finger gesture aborted; camera does not zoom or pan	✓ Pass
Release UI touch	Camera control restored on the next frame after finger lifts	✓ Pass
Rapid repeated UI taps	No stuck entries in `_uiTouches`; each tap registers and releases cleanly	✓ Pass
Desktop mouse over UI button	Camera does not orbit while holding mouse button over a button	✓ Pass
Desktop mouse leaving UI mid-drag	No camera jump on first drag frame after leaving UI	✓ Pass
Compile verification	Zero errors, zero warnings	✓ Pass

Lessons Learned

Position versus ownership

`IsPointerOverGameObject` answers a positional question that is only reliable at the moment a touch begins. Treating it as a reliable per-frame state during a drag is a category error. The correct abstraction is ownership: a touch either belongs to the UI or it does not, and that ownership is determined once and held.

Touch lifetime matters more than instantaneous position

A touch that begins on a UI element is a UI interaction for its entire duration. The user's intent — to interact with a slider, toggle, or button — does not change because their finger momentarily drifts outside the element's RectTransform. Modelling that intent as a lifetime property (a registered finger ID) rather than a frame-by-frame position produces correct behaviour across all gesture shapes and durations.

Separating input generation from camera movement

`TouchCameraInput` writes deltas; `OrbitCameraController` reads and applies them. These responsibilities being in separate classes, communicating through a cached value, means the gate can be applied entirely at the source without touching the consumer. The consumer does not need to be made aware of UI — it simply receives a zero delta when the gate is closed.

Investigate before implementing

The initial fix — adding `IsPointerOverGameObject` guards — was architecturally appropriate but tactically wrong because the root cause had not been fully established. The second iteration, which produced the correct fix, was possible only because the investigation identified the specific mechanism (`PointerExit` clearing `pointerEnter` during a drag) rather than stopping at the symptom. Diagnosing before coding eliminates the cost of implementing and then replacing a solution that addresses the wrong problem.

Conclusion

The camera input system now correctly distinguishes between viewport touches and UI touches for the entire lifetime of each contact, regardless of where the finger travels after it initially lands. The fix is localised to the one class responsible for producing touch-driven camera deltas, requires no changes to the camera controller or input aggregator, and introduces no runtime allocations. The implementation is explicit about the distinction between positional queries and ownership tracking, making it straightforward to maintain and extend if additional UI elements or gesture types are added in the future.